

Sprawozdanie z projektu

Verity Lens

System do ostrożnej weryfikacji informacji, wiarygodności newsów i ryzyka obrazów AI

Autor: Artsiom Markevich

Kierunek / grupa: *do uzupełnienia*

Prowadzący: *do uzupełnienia*

31 maja 2026

Repozytorium	Artsiom-beep/fake-news-detector
Backend publiczny	fake-news-detector-poi6.onrender.com
Ostatni commit opisany w raporcie	b99006d
Tryb produktu	PC desktop, Web UI, REST API, Android APK, Render cloud

Contents

Streszczenie	4
1 Cel projektu	4
1.1 Cel główny	4
1.2 Cele szczegółowe	4
1.3 Ostateczny zakres produktu	5
2 Punkt wyjścia	5
2.1 Stan początkowy	5
2.2 Pierwsza diagnoza	5
3 Co się nie udało i dlaczego zmieniono podejście	6
3.1 Stary klasyfikator fake/real	6
3.2 Telefon przez LAN i tunele tymczasowe	6
3.3 Niejasny status wersji na telefonie	6
3.4 Błędy narzędzia Codex Desktop	7
3.5 Zbyt zachowawcze odpowiedzi na proste fakty	7
3.6 Ocena newsów z dużych źródeł	7
4 Nowe podejście projektowe	7
4.1 Jeden kanoniczny silnik	7
4.2 Zasada ostrożności	8
4.3 Oddzielenie news credibility od fact-checkingu	8
5 Architektura końcowa	9
5.1 Warstwy systemu	9
5.2 Przepływ żądania	9
5.3 Kontrakt API	9
6 Implementacja interfejsów	10
6.1 Web UI	10
6.2 Aplikacja desktopowa Windows	10
6.3 Aplikacja mobilna Flutter	11
7 Wdrożenie chmurowe	11
7.1 Render	11
7.2 Dłaczego chmura była potrzebna	11
8 Testy i dowody działania	12
8.1 Testy automatyczne	12
8.2 Acceptance benchmark	12
8.3 Dowód działania na fizycznym telefonie	12
8.4 Paczka oddania	13
9 Historia prac	13

10 Najważniejsze decyzje projektowe	14
10.1 Nie używać starego ML jako głównego produktu	14
10.2 Nie obiecywać globalnej dokładności 95% bez benchmarku	14
10.3 Traktować telefon jako klienta, nie jako backend	14
10.4 Oddzielić demo od produkcji	14
10.5 Usunąć Screenshots z UI	14
11 Aktualny sposób działania	14
11.1 Na komputerze	14
11.2 Na telefonie	15
11.3 W repozytorium	15
12 Ograniczenia	15
13 Możliwe dalsze prace	15
14 Wnioski	16
A Załącznik A: Komendy reprodukcyjne	16
B Załącznik B: Najważniejsze artefakty	16
C Załącznik C: Skrócony opis dla uruchomienia	17

Streszczenie

Celem projektu było zbudowanie praktycznego systemu do weryfikacji informacji, który można uruchomić na komputerze, w przeglądarce oraz na telefonie. Projekt rozpoczął się jako klasyczny *fake news detector*, ale w trakcie prac okazało się, że prosta klasyfikacja *fake/real* jest niewystarczająca i często nieuczciwa wobec użytkownika. Ostateczny produkt, nazwany *Verity Lens*, nie próbuje zgadywać za wszelką cenę. System zwraca twardy werdykt *true* lub *fake* tylko wtedy, gdy posiada silną podstawę: prostą stabilną wiedzę, jawny werdykt fact-checkingu albo jednoznaczne reguły. Dla zwykłych artykułów informacyjnych zwracana jest ocena wiarygodności, a dla obrazów ocena ryzyka AI, nie absolutny dowód.

Raport opisuje cały przebieg projektu: stan początkowy, problemy ze starym modelem ML, porządkowanie architektury, zmianę podejścia na jeden kanoniczny silnik, budowę API, Web UI, aplikacji desktopowej Windows, aplikacji Flutter, wdrożenie Render, testy na telefonie oraz końcowe uproszczenie interfejsu przez usunięcie osobnej zakładki *Screenshots*. Układ raportu oparto na typowych zasadach sprawozdań projektowych: metadane, opis metody, wyniki i interpretacja, napotkane problemy oraz wnioski. Takie elementy pojawiają się m.in. w zaleceniach Politechniki Poznańskiej dla sprawozdań projektowych, materiałach Politechniki Warszawskiej o sprawozdaniach w LaTeX-u oraz w akademickich wymaganiach dotyczących raportów składanych w LaTeX-u [1, 2, 3].

1 Cel projektu

1.1 Cel główny

Celem głównym było przygotowanie działającego produktu, który pomaga użytkownikowi ocenić, czy informacja jest prawdziwa, fałszywa, niepewna albo czy artykuł wygląda wiarygodnie. Produkt miał być możliwy do pokazania w praktyce: lokalnie na PC, jako paczka desktopowa Windows, jako API, jako aplikacja mobilna oraz jako rozwiązanie działające przez internet bez konieczności trzymania komputera włączonego.

1.2 Cele szczegółowe

- uporządkować projekt po wcześniejszych eksperymentach ML i usunąć konflikt wielu wejść produktowych;
- stworzyć jeden kanoniczny silnik decyzyjny dla CLI, API, UI, desktopu i telefonu;
- rozdzielić zwykle sprawdzanie faktów od oceny wiarygodności newsów;
- uniknąć fałszywej pewności tam, gdzie brakuje źródeł;
- uruchomić backend lokalnie i w chmurze Render;
- zbudować aplikację mobilną Flutter z publicznym API;
- zbudować wersję Windows portable;
- przygotować testy automatyczne i paczkę oddania z dowodami działania.

1.3 Ostateczny zakres produktu

W aktualnej wersji użytkownik widzi trzy główne moduły:

- **News** – ocena wiarygodności artykułu lub adresu URL;
- **Facts** – sprawdzanie krótkiego twierdzenia;
- **Images** – ocena ryzyka, że obraz został wygenerowany lub przetworzony przez AI.

Funkcja OCR dla zrzutów ekranu pozostała w backendzie oraz w testach regresyjnych jako `screenshot_ocr`, ale nie jest już osobną zakładką w interfejsie użytkownika. Decyzja ta została podjęta po testach użyteczności: zakładka *Screenshots* wyglądała jak pełnoprawna funkcja dla użytkownika, a w praktyce była mniej jasna niż podstawowe moduły i zwiększała zamieszanie w aplikacji mobilnej.

2 Punkt wyjścia

2.1 Stan początkowy

Na początku projekt był mieszanką kilku podejść:

- archiwalny model ML do klasyfikacji *fake news*;
- prototypowe pliki predykcji i treningu;
- kilka potencjalnych punktów wejścia API;
- brak jednego jasnego kontraktu dla aplikacji mobilnej;
- brak pewności, która część kodu jest produktem, a która eksperymentem.

Największym problemem nie był brak kodu, tylko brak jednej prawdy architektonicznej. Istniał stary model, który mógł wyglądać atrakcyjnie jako “AI detector”, ale był zbyt ciężki, słabo kontrolowalny i nie nadawał się jako pewny produkt do demonstracji. Pojawiał się też ryzykowny problem semantyczny: system mógł zachowywać się jak klasyfikator, który na siłę wybiera *fake* albo *real*, nawet gdy dane nie dawały do tego podstaw.

2.2 Pierwsza diagnoza

Po analizie projektu przyjęto trzy założenia:

1. produkt musi mieć jeden kanoniczny silnik, a nie kilka konkurencyjnych ścieżek;
2. wynik *uncertain* jest poprawnym wynikiem, jeżeli system nie posiada mocnych dowodów;
3. zwykły artykuł newsowy nie jest tym samym co twierdzenie sprawdzane przez fact-checking.

To zmieniło kierunek prac. Zamiast ulepszać stary klasyfikator, projekt został przebudowany wokół ostrożnego pipeline’u, który potrafi powiedzieć: “nie wiem wystarczająco dużo”.

3 Co się nie udało i dlaczego zmieniono podejście

3.1 Stary klasyfikator fake/real

Pierwotny kierunek opierał się na myśleniu: użytkownik podaje tekst, a model zwraca *fake* albo *real*. To podejście nie spełniało wymagań produktu z kilku powodów:

- model był ciężki i niewygodny do pakowania w desktop oraz chmurę;
- wynik klasyfikatora nie wyjaśniał użytkownikowi, na czym opiera się decyzja;
- system nie odróżniał artykułu newsowego od pojedynczego twierdzenia;
- fałszywa pewność była większym ryzykiem niż ostrożna odpowiedź **uncertain**.

Dlatego stary kod ML został oddzielony od runtime’u produktu. Nie został bezmyślnie usunięty, lecz przeniesiony do archiwum, aby zachować historię i możliwość analizy:

- `archive/legacy_multimodal/`
- `archive/legacy_factcheck_v1/`
- `docs/legacy_ml_inventory.md`

3.2 Telefon przez LAN i tunele tymczasowe

Na etapie mobilnym początkowo wystarczało, że telefon łączył się z komputerem w tej samej sieci Wi-Fi albo przez tymczasowy tunel. To działało do demonstracji, ale nie rozwiązywało głównego celu: telefon miał działać bez komputera. Tryb LAN wymagał, aby PC był włączony, API działało lokalnie, firewall przepuszczał ruch, a telefon był w tej samej sieci. Tunel tymczasowy również był niewystarczający, bo adres mógł wygasnąć.

Rozwiązaniem stało się wdrożenie backendu na Render i budowa APK z osadzonym publicznym adresem HTTPS:

`https://fake-news-detector-poi6.onrender.com`

3.3 Niejasny status wersji na telefonie

W trakcie testów pojawił się praktyczny problem: po zbudowaniu nowej aplikacji telefon nadal potrafił pokazywać stary interfejs. W szczególności po usunięciu zakładki *Screenshots* na ekranie nadal była widoczna stara wersja. Problem rozwiązano przez:

- czyszczenie buildu Flutter poleceniem `flutter clean`;
- ponowne pobranie zależności;
- czystą kompilację APK release;
- odinstalowanie starego pakietu z telefonu;
- ponowną instalację aktualnego `VerityLens-cloud.apk`;
- sprawdzenie pakietu `app.veritylens.mobile` przez ADB.

3.4 Błędy narzędzia Codex Desktop

Podczas prac pojawiały się okna błędu typu *Unable to find Electron app*. Po analizie uznano, że nie był to błąd projektu Verity Lens, backendu, Pythona, Fluttera ani telefonu. Był to problem środowiska narzędziowego aplikacji Codex Desktop, prawdopodobnie związany z obsługą wewnętrznych akcji `click/action`. Nie wpłynęło to na produkt, ale wprowadzało zamieszanie podczas pracy.

3.5 Zbyt zachowawcze odpowiedzi na proste fakty

Jednym z praktycznych przykładów był fakt typu:

Books are made out of paper.

System początkowo odpowiadał zbyt ostrożnie, mimo że dla takiego stabilnego, codziennego faktu można zastosować lokalną wiedzę i prostsze reguły. Dodano obsługę typowych faktów materiałowych, dzięki czemu proste twierdzenia nie muszą przechodzić przez pełny ciężki pipeline źródłowy.

3.6 Ocena newsów z dużych źródeł

Dla zwykłych artykułów BBC lub podobnych źródeł system początkowo bywał zbyt nieśmiały. Problem polegał na tym, że brak wielu niezależnych potwierdzeń nie powinien automatycznie obniżać wiarygodnego artykułu do słabej oceny. Wprowadzono model `source_quality_corroboration_v2`, który osobno traktuje:

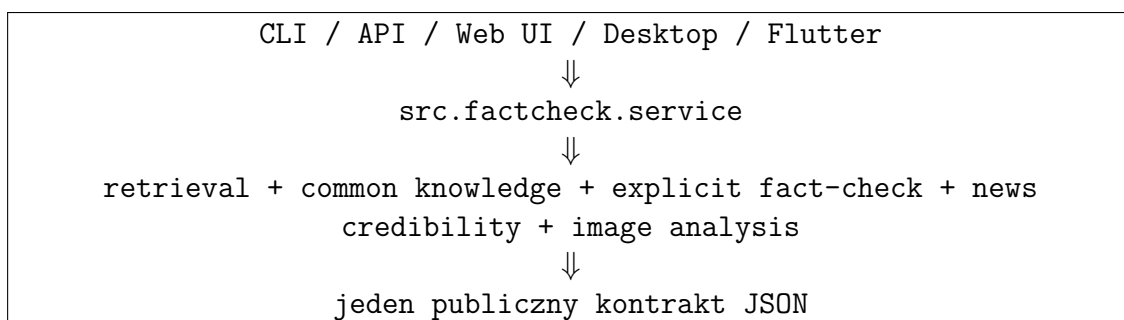
- jakość źródła;
- jakość samego artykułu;
- niezależne potwierdzenia;
- flagi ryzyka.

Dzięki temu dobry artykuł z wiarygodnej redakcji może otrzymać ocenę wysoką albo średnią bez udawania, że system dowiódł prawdziwości każdej informacji zawartej w artykule.

4 Nowe podejście projektowe

4.1 Jeden kanoniczny silnik

Najważniejszą zmianą architektoniczną było utworzenie jednej ścieżki produktu:



W praktyce oznacza to, że aplikacja na telefonie nie ma własnej logiki decyzyjnej. Telefon wysyła żądanie do API, a wynik jest liczony przez ten sam backend, którego używa Web UI i desktop. Zmniejsza to ryzyko, że różne wersje produktu będą odpowiadały inaczej na to samo pytanie.

4.2 Zasada ostrożności

W projekcie przyjęto zasadę:

Jeżeli system nie ma wystarczających dowodów, powinien powiedzieć **uncertain**, a nie zgadywać.

To jest decyzja produktowa, nie tylko techniczna. System do weryfikacji informacji może zaszkodzić użytkownikowi, jeżeli zbyt łatwo nada etykietę *fake*. Dlatego *Verity Lens* ma trzy poziomy odpowiedzi:

- **true** – mocne potwierdzenie lub stabilna wiedza;
- **fake** – mocne obalenie lub jawny werdykt fact-checkingu;
- **uncertain** – brak wystarczającego dowodu.

4.3 Oddzielenie news credibility od fact-checkingu

Artykuł informacyjny nie jest tym samym co twierdzenie typu “Słońce jest gwiazdą”. Dlatego dla modułu *News* wynik opiera się głównie na polu **credibility**, a nie na twardym **true/fake**. System wylicza:

- **source_score** – zaufanie do domeny;
- **article_quality_score** – jakość struktury artykułu;
- **corroboration_score** – potwierdzenia z niezależnych źródeł;
- **risk_score** – flagi ryzyka, np. brak daty albo słaba struktura.

Taki wynik jest uczciwszy: mówi, czy artykuł wygląda wiarygodnie, ale nie udaje pełnego dowodu logicznego.

5 Architektura końcowa

5.1 Warstwy systemu

Warstwa	Najważniejsze pliki	Rola
Core engine	<code>src/factcheck/service.py</code>	Orkiestracja głównego pipeline'u i wybór strategii.
Decision layer	<code>src/factcheck/decision.py</code>	Decyzja <code>true/fake/uncertain</code> .
Retrieval	<code>src/factcheck/retrieval.py</code>	Pobieranie i dobór materiału źródłowego.
Common knowledge	<code>src/factcheck/common_knowledge.py</code>	Proste fakty, reguły i lokalna wiedza.
News credibility	<code>src/factcheck/news_credibility.py</code>	Punktacja artykułów informacyjnych.
Image/OCR	<code>src/factcheck/image_analysis.py</code>	OCR i ocena ryzyka obrazu AI.
REST API	<code>src/api_factcheck.py</code>	Publiczne endpointy dla Web UI i Fluttera.
Web UI	<code>src/ui.py</code>	Interfejs przeglądarkowy.
Desktop	<code>src/desktop_app.py</code>	PyWebView i lokalny serwer w aplikacji Windows.
Mobile	<code>apps/fake_news_detector_flutter/</code>	Aplikacja Flutter dla telefonu.
Cloud	<code>Dockerfile</code> , <code>render.yaml</code>	Wdrożenie backendu na Render.

5.2 Przepływ żądania

1. Użytkownik wpisuje URL, twierdzenie albo wybiera obraz.
2. Interfejs wysyła żądanie do API.
3. API waliduje dane wejściowe.
4. Silnik wybiera odpowiednią ścieżkę: `fact-check`, `common knowledge`, `news credibility` albo `image analysis`.
5. Wynik jest zwracany jako JSON.
6. UI renderuje krótką odpowiedź, szczegóły, źródła i ostrzeżenia.

5.3 Kontrakt API

Publiczny kontrakt zawiera cztery najważniejsze endpointy:

```
GET /health
GET /ready
POST /factcheck
POST /factcheck-image
```

Przykładowe pola wyniku:

```
verdict: true | fake | uncertain
confidence: liczba 0.0-1.0
summary: kr\ 'otkie wyja\ 'snienie
```

```
evidence: lista \'zr\'odeł{}
credibility: ocena wiarygodno\'sci news\'ow
image_analysis: OCR albo ocena ryzyka AI
trace: diagnostyka pipeline'u
```

Stabilność tego kontraktu jest ważna, ponieważ korzysta z niego aplikacja Flutter.

6 Implementacja interfejsów

6.1 Web UI

Interfejs webowy został zaimplementowany w `src/ui.py`. W aktualnej wersji widoczne są trzy sekcje:

- News;
- Facts;
- Images.

Sekcja *Screenshots* została usunięta z UI. Backendowa funkcja OCR nadal istnieje, ale nie jest osobnym modułem użytkownika. Po tej zmianie interfejs jest prostszy i lepiej odpowiada rzeczywistym zastosowaniom.

6.2 Aplikacja desktopowa Windows

Wersja desktopowa nie jest osobnym produktem logicznym. Jest opakowaniem lokalnego Web UI w okno aplikacji Windows. Działa następująco:

1. `FakeNewsDetector.exe` uruchamia lokalny serwer Uvicorn.
2. Serwer publikuje `src.ui:app` na wolnym porcie localhost.
3. PyWebView otwiera lokalny adres w natywnym oknie.
4. Po zamknięciu okna serwer jest zatrzymywany.

Aktualna paczka desktopowa:

- `dist/FakeNewsDetector/FakeNewsDetector.exe`
- `dist/FakeNewsDetector-Windows-Portable.zip`

Po ostatniej przebudowie paczki desktopowej:

- EXE SHA256: 273110CFC51015EC5924575B840A1A907B8963F05280195C9C8A9F0884F13FD0;
- ZIP SHA256: 6B86B6FDA42937C7AE252700C6DA653A9E7ADA6A6E8D8C94DA25BA995C4313C0.

6.3 Aplikacja mobilna Flutter

Aplikacja mobilna jest klientem API. Nie uruchamia Pythona na telefonie. Schemat pracy wygląda tak:

telefon -> internet -> Render API -> wynik -> telefon

Pakiet Android:

- identyfikator: `app.veritylens.mobile`;
- główna aktywność: `app.veritylens.mobile.MainActivity`;
- publiczny backend: <https://fake-news-detector-poi6.onrender.com>;
- aktualny cloud APK po czystej przebudowie: `outputs/phone_download/VerityLens-cloud.apk`;
- APK SHA256: 51DE2478B9C34E6CFA6CBD910E135758
E56DC4D5F9DEF9FE2733BA7D4DB32E12.

Telefon działa bez komputera, ale wymaga internetu i działającego backendu Render. Wersja darmowa Render może zasypiać, dlatego pierwszy request może trwać kilkadziesiąt sekund. To ograniczenie platformy hostingowej, nie błąd aplikacji.

7 Wdrożenie chmurowe

7.1 Render

Backend został wdrożony jako web service na Render. Render buduje i uruchamia aplikację z repozytorium GitHub lub z Dockerfile, co odpowiada dokumentacji platformy dotyczącej usług webowych i Docker runtime [7, 8]. W projekcie zastosowano lekki runtime API:

- `requirements.api.txt`;
- `Dockerfile`;
- `render.yaml`.

Z paczki chmurowej wykluczono:

- stare modele treningowe;
- foldery `build/dist/outputs`;
- lokalne środowisko `.venv`;
- ciężkie opcjonalne zależności Torch/Transformers.

7.2 Dlaczego chmura była potrzebna

Telefon z aplikacją Flutter nie powinien wymagać komputera użytkownika. Dlatego sam APK nie wystarcza. Potrzebny jest publiczny backend HTTPS. Po wdrożeniu na Render telefon może wysłać żądanie z dowolnej sieci z dostępem do internetu.

8 Testy i dowody działania

8.1 Testy automatyczne

Aktualny stan po ostatnich pełnych kontrolach:

Zakres	Narzędzie / komenda	Wynik
Backend unit/integration	<code>unittest</code>	116/116 OK
Product acceptance	<code>run_product_acceptance.py</code>	93/93 OK
Flutter analyze	<code>flutteranalyze</code>	No issues found
Flutter tests	<code>fluttertest</code>	11/11 OK
Release gate	<code>release gate</code>	OK, 25 kroków w ostatnim trybie cloud/phone
Final cloud/phone	<code>finalizer cloud-phone</code>	OK
Desktop package	<code>verifier desktopowy</code>	OK

8.2 Acceptance benchmark

Zamiast deklarować ogólną skuteczność na całym internecie, projekt ma mierzalną bramkę regresyjną. Ostatni wynik acceptance benchmark:

Sekcja	Poprawne	Wszystkie	Skuteczność
Facts	44	44	100%
News	15	15	100%
Screenshot OCR API	12	12	100%
Images	11	11	100%
API/mobile contract	11	11	100%
Razem	93	93	100%

Należy to interpretować poprawnie: wynik 93/93 oznacza, że aktualny produkt przechodzi zdefiniowane przypadki kontrolne. Nie oznacza to matematycznej gwarancji 100% dla wszystkich możliwych tekstów i newsów w internecie. To jednak dobry fundament inżynierski, bo każda przyszła zmiana może zostać porównana z tym samym zestawem regresji.

8.3 Dowód działania na fizycznym telefonie

Fizyczny telefon został wykryty przez ADB jako:

RFCTC07YX2N, model SM-G781B

W końcowym wrapperze wymagano prawdziwego urządzenia, a nie emulatora. Raporty potwierdziły:

- urządzenie jest fizyczne;
- APK cloud jest w trybie release;
- w APK osadzono publiczny adres Render;
- telefon potrafi wykonać smoke test;
- zrzut ekranu jest poprawnym plikiem PNG;
- SHA256 zrzutu zgadza się z plikiem.

8.4 Paczka oddania

Wygenerowano paczkę:

`outputs/submission/VerityLens-submission.zip`

Weryfikator paczki sprawdził:

- obecność raportów końcowych;
- obecność desktop ZIP;
- obecność APK;
- czysty ZIP źródół;
- brak zabronionych plików lokalnych lub wygenerowanych w źródłach;
- zgodność raportów z artefaktami.

Po końcowym audycie projekt miał status:

complete = true, estimated completion = 100%, remaining = 0%

9 Historia prac

Etap	Co robiono	Najważniejszy efekt
Analiza projektu	Przegląd istniejących plików, modeli i wejść API.	Wykryto konflikt między prototypem ML a produktem.
Refaktoryzacja	Utworzenie kanonicznego pipeline'u i archiwizacja legacy.	Jeden silnik dla API, UI, desktopu i telefonu.
Ostrożność decyzyjna	Wprowadzenie zasady uncertain .	System nie wymusza fałszywej pewności.
Facts	Dodanie reguł prostych faktów, arytmetyki i wiedzy lokalnej.	Lepsze odpowiedzi na stabilne codzienne twierdzenia.
News	Oddzielenie credibility od hard verdict.	Artykuły newsowe są oceniane jako wiarygodne/średnie/słabe, nie jako automatycznie prawdziwe albo fałszywe.
Images	Dodanie risk assessment dla AI obrazów.	Wynik mówi o ryzyku, nie o absolutnym dowodzie.
Web UI	Zbudowanie oddzielnych paneli produktu.	Użytkownik może pracować przez przeglądarkę.
Desktop	PyInstaller + PyWebView.	Powstała paczka Windows portable.
Mobile	Flutter + API client.	Powstał APK Android.
Chmura	Render + Docker + release scripts.	Telefon może działać bez PC.
Dowody	Release gate, smoke tests, verifier, raporty.	Projekt ma powtarzalne potwierdzenie działania.
Uproszczenie UI	Usunięcie widocznej zakładki Screenshots.	Interfejs ma trzy jasne funkcje: News, Facts, Images.

10 Najważniejsze decyzje projektowe

10.1 Nie używać starego ML jako głównego produktu

Ta decyzja była kluczowa. Stary model mógł dawać szybkie etykiety, ale nie dawał zaufanego produktu. W projekcie weryfikacji informacji lepsza jest odpowiedź ostrożna i uzasadniona niż pewna, ale niekontrolowana.

10.2 Nie obiecywać globalnej dokładności 95% bez benchmarku

W rozmowie projektowej pojawiał się cel wysokiej jakości. Zamiast deklarować pustą liczbę, utworzono *product acceptance gate*. Dzięki temu 95% jest sprawdzane na konkretnych sekcjach i przypadkach, a nie tylko wpisane w opis.

10.3 Traktować telefon jako klienta, nie jako backend

Uruchamianie całego Pythona i OCR bezpośrednio na telefonie byłoby kosztowne i niestabilne. Dlatego aplikacja mobilna wysyła żądania do chmury. To upraszcza APK, ale wymaga internetu.

10.4 Oddzielić demo od produkcji

Tryb LAN i tymczasowy tunnel są dobre do debugowania. Nie są jednak dowodem, że aplikacja działa sama na telefonie. Dlatego skrypty rozróżniają:

- LAN APK;
- internet tunnel APK;
- cloud APK release dla stałego Render URL.

10.5 Usunąć Screenshots z UI

Ostatnia decyzja produktowa dotyczyła uproszczenia. Funkcja screenshot OCR działa technicznie, ale jako oddzielna zakładka była mało użyteczna dla użytkownika końcowego. Została więc ukryta w API i testach regresyjnych, a interfejs końcowy ma tylko trzy czytelne moduły.

11 Aktualny sposób działania

11.1 Na komputerze

Użytkownik może uruchomić:

- `dist/FakeNewsDetector/FakeNewsDetector.exe` – gotowa aplikacja desktopowa;
- `run_desktop_app.bat` – lokalny wrapper;
- `python -m uvicorn src.ui:app -port 8002` – Web UI;
- `python -m uvicorn src.api_factcheck:app -port 8001` – samo API.

11.2 Na telefonie

Telefon z aktualnym APK działa bez komputera, jeżeli ma internet. Schemat:

Verity Lens APK -> <https://fake-news-detector-poi6.onrender.com> ->
FastAPI -> wynik

Jeżeli Render Free zasnął, aplikacja może przez chwilę pokazać *API not reachable*. Po wybudzeniu serwera odświeżenie statusu powinno przywrócić połączenie.

11.3 W repozytorium

Zmiany zostały wypchnięte na GitHub. Najważniejsze commity końcowe:

Commit	Znaczenie
7c57851	Początkowe uporządkowanie projektu Verity Lens.
fb97373	Finalizacja dowodów cloud/phone.
d5d0e16	Obsługa prostych faktów materiałowych.
ade5984	Kalibracja wiarygodności newsów.
892555a	Odświeżenie dowodów po kalibracji newsów.
b99006d	Usunięcie zakładki Screenshots z UI aplikacji.

12 Ograniczenia

- Backend w Render Free może zasypiać, więc pierwszy request może być wolny.
- Telefon nie działa offline, bo silnik działa w Pythonie po stronie backendu.
- Ocena AI obrazu jest oceną ryzyka, nie dowodem sądowym.
- OCR może się mylić przy zrzutach niskiej jakości, dlatego funkcja została wycofana z głównego UI.
- Obecny benchmark jest dobry jako regresja produktu, ale nie reprezentuje całego internetu.
- Aktualne fakty polityczne i urzędowe mogą się dezaktualizować, więc wymagają okresowej aktualizacji źródeł.

13 Możliwe dalsze prace

- dodać ekran *About / Version* w aplikacji mobilnej, aby od razu było widać commit, wersję APK i adres API;
- poprawić baner połączenia tak, aby przy Render Free pokazywał *Waking up server* i automatycznie ponawiał próbę;
- rozszerzyć benchmark acceptance o większy zbiór żywych newsów i trudnych przypadków;
- przygotować tryb demonstracyjny z zapisanymi przykładami wejściowymi;
- dodać historię ostatnich sprawdzeń w aplikacji mobilnej;
- rozważyć płatny hosting, jeżeli aplikacja ma działać szybko i stale.

14 Wnioski

Projekt zakończył się działającym produktem, a nie tylko prototypem. Najważniejszym sukcesem jest zmiana podejścia: od starego klasyfikatora *fake/real* do ostrożnego systemu weryfikacji informacji. Produkt ma jeden silnik, stabilne API, Web UI, desktop Windows, aplikację Flutter i backend w chmurze. Został przetestowany automatycznie i praktycznie, w tym na fizycznym telefonie.

Najważniejsza lekcja projektowa brzmi: w systemach fact-checkingowych brak pewności nie jest porażką. Porażką byłoby udawanie pewności. Dlatego *Verity Lens* jest zaprojektowany tak, aby pomagać użytkownikowi myśleć ostrożniej, a nie zastępować myślenie czarną skrzynką.

A Załącznik A: Komendy reprodukcyjne

```
python -m unittest discover -s tests -v
python scripts/run_product_acceptance.py
.\scripts\build_report.ps1
.\scripts\build_windows.ps1
.\scripts\verify_desktop_package.ps1
.\scripts\run_release_gate.ps1
.\scripts\build_phone_for_cloud.ps1 '
  -ApiBaseUrl https://fake-news-detector-poi6.onrender.com '
  -Mode release
.\scripts\finalize_cloud_phone_submission.ps1 '
  -ApiBaseUrl https://fake-news-detector-poi6.onrender.com
flutter analyze
flutter test
adb devices
```

B Załącznik B: Najważniejsze artefakty

Artefakt	Ścieżka
Raport LaTeX	docs/report/verity_lens_report.tex
Raport PDF	docs/report/verity_lens_report.pdf
Desktop EXE	dist/FakeNewsDetector/FakeNewsDetector.exe
Desktop ZIP	dist/FakeNewsDetector-Windows-Portable.zip
Cloud APK	outputs/phone_download/VerityLens-cloud.apk
Submission ZIP	outputs/submission/VerityLens-submission.zip
Product acceptance report	reports/product_acceptance_latest.json
Release gate report	reports/release_gate_latest.json
Final cloud phone report	reports/final_cloud_phone_submission_latest.json

C Załącznik C: Skrócony opis dla uruchomienia PC

Uruchomić:

```
dist/FakeNewsDetector/FakeNewsDetector.exe
```

Telefon

Zainstalować:

```
outputs/phone_download/VerityLens-cloud.apk
```

Telefon potrzebuje internetu. Backend działa pod adresem:

<https://fake-news-detector-poi6.onrender.com>

References

- [1] Politechnika Poznańska, Instytut Informatyki. *Sprawozdanie – wymagania dla projektu*. [strona kursu i wymagań sprawozdania](#)
- [2] T. J. Kruk, Politechnika Warszawska. *Przygotowywanie sprawozdania w systemie LaTeX*. [materiały o sprawozdaniach w LaTeX-u](#)
- [3] Uniwersytet Gdański. *Matematyka obliczeniowa – profil informatyczny: wymagania dotyczące sprawozdań w LaTeX-u*. [sylabus i wymagania raportowe](#)
- [4] Politechnika Poznańska. *Projekt hurtowni danych – sprawozdanie, opis problemów i rozwiązań*. [opis zaliczenia projektu](#)
- [5] FastAPI. *Features: automatic docs and OpenAPI*. [oficjalna dokumentacja FastAPI](#)
- [6] Flutter Documentation. *Build and release an Android app*. [oficjalna dokumentacja Flutter Android](#)
- [7] Render Documentation. *Docker on Render*. [oficjalna dokumentacja Render Docker](#)
- [8] Render Documentation. *Your First Render Deploy*. [oficjalna dokumentacja Render deploy](#)